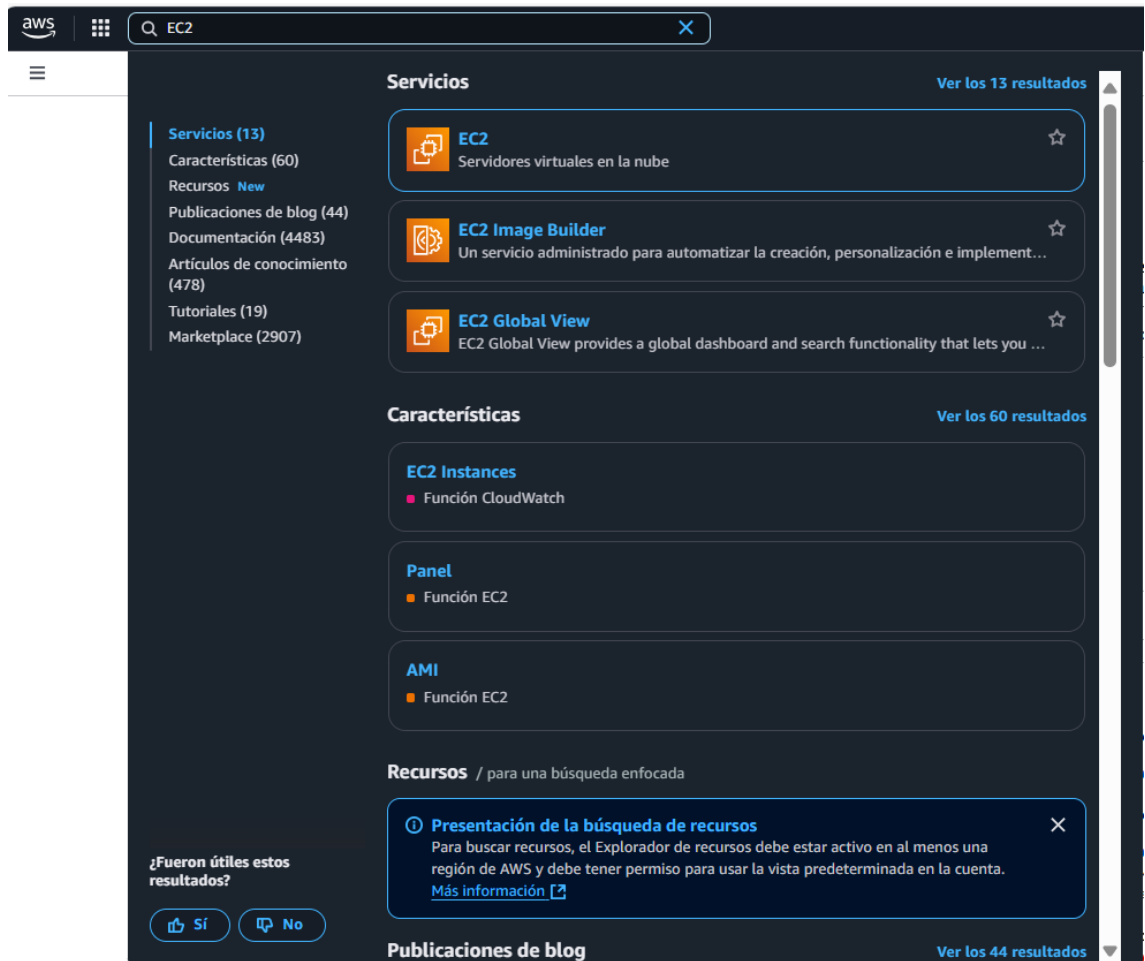
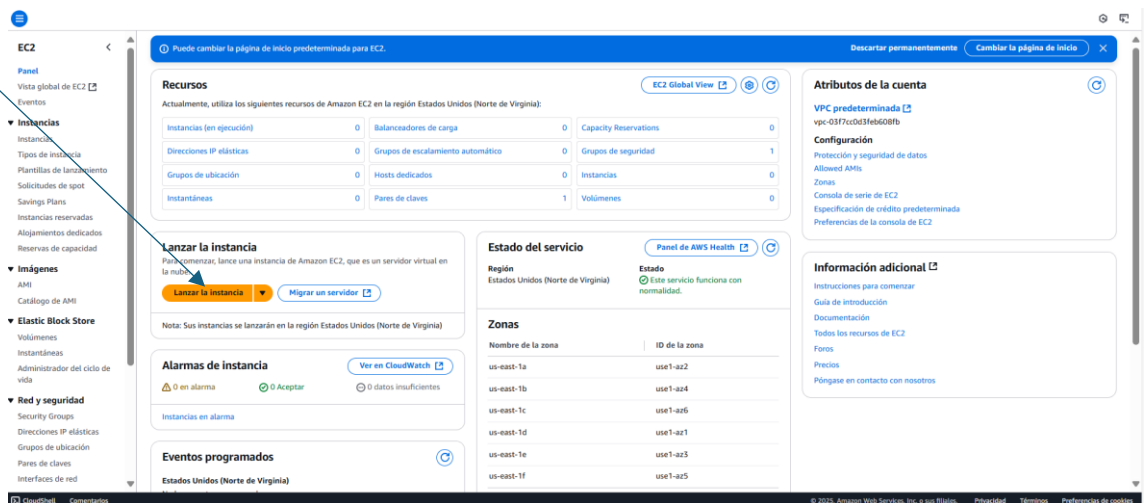


PRACTICA 8 APACHE KAFKA MEDIANTE USO DE UNA INSTANCIA DE EC2-EN AMAZON

1. Entrar en el servicio EC2



2. Pulsar botón Lanzar Instancia



3. Nos aparece la ventana de configuración de la instancia, aquí debemos añadir lo siguiente:

- a. Nombre y AMI de imagen. Ponemos el nombre que queramos y seleccionamos la AMI de AWS Linux que aparece por defecto

parctica Kafka [Agregar etiquetas adicionales](#)

▼ **Imágenes de aplicaciones y sistemas operativos (Imagen de máquina de Amazon)** [Información](#)

Una AMI es una plantilla que contiene la configuración de software (sistema operativo, servidor de aplicaciones y aplicaciones) necesaria para lanzar la instancia. Busque o examine las AMI si no ve lo que busca a continuación.

Q Busque en nuestro catálogo completo que incluye miles de imágenes de sistemas operativos y aplicaciones

Mis AMI [Inicio rápido](#)

Amazon Linux macOS Ubuntu Windows Red Hat SUSE Linux Debian

Imágenes de máquina de Amazon (AMI)

AMI de Amazon Linux 2023
ami-0f88e0871f081e91 (64 bits (x86), uefi-preferred) / ami-0bc72bd3a8ba059d (64 bits (Arm), uefi)
Virtualización: hvm Activado para ENA: true Tipo de dispositivo raíz: ebs

Apto para la capa gratuita

Descripción
Amazon Linux 2023 es un sistema operativo moderno y de uso general basado en Linux que incluye 5 años de soporte diseñado para proporcionar un entorno de ejecución seguro, estable y de alto desempeño para desarrollar y ejecutar sus aplicaciones en la nube.

Amazon Linux 2023 AMI 2023.7.20250428.1 x86_64 HVM kernel-6.1

Arquitectura: 64 bits (x86) Modo de arranque: uefi-preferred ID de AMI: ami-0f88e0871f081e91 Fecha de publicación: 2025-04-30 Nombre de usuario: ec2-user [Proveedor verificado](#)

▼ **Resumen**

Número de instancias: 1 [Información](#)

Imagen de software (AMI)
Amazon Linux 2023 AMI 2023.7.2... [más información](#)
ami-0f88e0871f081e91

Tipo de servidor virtual (tipo de instancia)
t2.micro

Firewall (grupo de seguridad)
Nuevo grupo de seguridad

Almacenamiento (volúmenes)
Volúmenes: 1 (8 GiB)

ⓘ Nivel gratuito: Durante el primer año que abra una cuenta de AWS, obtiene 750 horas al mes de uso de instancias t2.micro (o t3.micro cuando t2.micro no esté disponible) si se utiliza con AMI de nivel gratuito, 750 horas al mes de uso de direcciones IPv4 públicas, 30 GiB de almacenamiento de EBS, 2 millones de E/S, 1 GiB de instantáneas y 100 GiB de ancho de banda para Internet.

[Cancelar](#) [Lanzar instancia](#) [Código de versión preliminar](#)

- b. Tipo de Instancia. Dejamos la opción predefinida de **t2.large**

▼ **Tipo de instancia** [Información](#) [Obtener asesoramiento](#)

Tipo de instancia

t2.micro [Apto para la capa gratuita](#)

Familia: t2 1 vCPU 1 GiB Memoria Generación actual: true

Bajo demanda Windows base precios: 0.0162 USD por hora Bajo demanda Ubuntu Pro base precios: 0.0134 USD por hora

Bajo demanda SUSE base precios: 0.0116 USD por hora Bajo demanda RHEL base precios: 0.026 USD por hora

Bajo demanda Linux base precios: 0.0116 USD por hora

[Se aplican costos adicionales a las AMI con software preinstalado](#)

☐ Todas las generaciones [Comparar tipos de instancias](#)

- c. Par de claves. Pulsamos botón generar par de claves.

▼ **Par de claves (inicio de sesión)** [Información](#)

Puede utilizar un par de claves para conectarse de forma segura a la instancia. Asegúrese de que tiene acceso al par de claves seleccionado antes de lanzar la instancia.

Nombre del par de claves - obligatorio

[Seleccionar](#) [Crear un nuevo par de claves](#)

- d. En la pantalla que aparece , indicamos el nombre del par y dejamos seleccionado .pen (para conectarnos vía ssh)

Crear par de claves

Nombre del par de claves

Con los pares de claves es posible conectarse a la instancia de forma segura.

maquina-david

El nombre puede incluir hasta 255 caracteres ASCII. No puede incluir espacios al principio ni al final.

Tipo de par de claves

RSA

Par de claves pública y privada cifradas mediante RSA

ED25519

Par de claves privadas y públicas cifradas ED25519

Formato de archivo de clave privada

.pem

Para usar con OpenSSH

.ppk

Para usar con PuTTY

Cuando se le solicite, almacene la clave privada en un lugar seguro y accesible del equipo. Lo necesitará más adelante para conectarse a la instancia. [Más información](#)

Cancelar

Crear par de claves

e. Ahora se selecciona par de claves

Par de claves (inicio de sesión)

Información

Puede utilizar un par de claves para conectarse de forma segura a la instancia. Asegúrese de que tiene acceso al par de claves seleccionado antes de lanzar la instancia.

Nombre del par de claves - obligatorio

maquina-david

Crear un nuevo par de claves

f. Pulsamos botón Lanzar Instancia y dejamos que se ponga activa (en servicio)

▼ Resumen

Número de instancias | [Información](#)

1

Imagen de software (AMI)

Amazon Linux 2023 AMI 2023.7.2...[más información](#)

ami-0f88e80871fd81e91

Tipo de servidor virtual (tipo de instancia)

t2.micro

Firewall (grupo de seguridad)

Nuevo grupo de seguridad

Almacenamiento (volúmenes)

Volúmenes: 1 (8 GiB)

i Nivel gratuito: Durante el primer año que abre una cuenta de AWS, obtiene 750 horas al mes de uso de instancias t2.micro (o t3.micro cuando t2.micro no esté disponible) si se utiliza con AMI de nivel gratuito, 750 horas al mes de uso de direcciones IPv4 públicas, 30 GiB de almacenamiento de EBS, 2 millones de E/S, 1 GB de instantáneas y 100 GB de ancho de banda para Internet.



[Cancelar](#)

[Lanzar instancia](#)

[Código de versión preliminar](#)

Correcto
El lanzamiento de la instancia se inició correctamente (i-0caufe1afa13b44db)

► Registro de lanzamiento

Pasos siguientes

Crear alertas de uso del nivel gratuito y facturación

Para administrar los costos y evitar facturas sorpresa, configure las notificaciones por correo electrónico para los umbrales de uso del nivel gratuito y facturación.

[Crear alertas de facturación](#)

Conectarse a la instancia

Una vez que la instancia esté en ejecución, inicie sesión en ella desde el equipo local.

[Conectarse a la instancia](#)

[Más información](#)

Conectar una base de datos de RDS

Configure la conexión entre una instancia de EC2 y una base de datos para permitir el flujo de tráfico entre ellas.

[Conectar una base de datos de RDS](#)

[Crear una nueva base de datos de RDS](#)

[Más información](#)

[Ver todas las instancias](#)

- | Instancias (1) Información | | | | | | | | | |
|--|----------------|---------------------|---|-------------------|---|--------------------------|------------------------|-------------------------|----------------------|
| <input type="text" value="Buscar instancia por atributo o etiqueta (case-sensitive)"/> | | | | Todos los ... ▾ | Última actualización
Hace less than a minute | | | | |
| | | | | | Conectar | Estado de la instancia ▾ | Acciones ▾ | Lanzar instancias ▾ | |
| ☐ | Name ↗ | ID de la instancia | Estado de la instancia | Tipo de instancia | Comprobación de estado | Estado de la instancia | Zona de disponibilidad | DNS de IPv4 pública | Dirección IP privada |
| ☐ | parcitra Kafka | i-0caafe1a8a13b448e | En ejecución | t2.micro | Inicializando | Ver alarmas + | us-east-1a | ec2-18-205-23-164.co... | 18.205.23.164 |

Resumen de instancia de i-Ocaafe1a8a13b448e (parctica Kafka)

Se ha actualizado hace less than a minute

ID de la instancia

i-Ocaafe1a8a13b448e

Dirección IPv6

-

Tipo de nombre de anfitrión

Nombre de IP: ip-172-31-94-3.ec2.internal

Responder al nombre DNS de recurso privado

IPv4 (A)

Dirección IP asignada automáticamente

18.205.23.164 [IP pública]

Rol de IAM

-

IMDSv2

Required

Operador

-

Dirección IPv4 pública

18.205.23.164 | dirección abierta

Estado de la instancia

En ejecución

Nombre DNS de IP privada (solo IPv4)

ip-172-31-94-3.ec2.internal

Tipo de instancia

t2.micro

ID de VPC

vpc-03f7cc0d3feb608fb

ID de subred

subnet-07a1bc82046aff9a

ARN de instancia

arnaws:ec2:us-east-1:441173487216:instance/i-Ocaafe1a8a13b448e

Direcciones IPv4 privadas

172.31.94.3

DNS de IPv4 pública

ec2-18-205-23-164.compute-1.amazonaws.com | dirección abierta

Direcciones IP elásticas

-

Hallazgo de AWS Compute Optimizer

Suscribirse a AWS Compute Optimizer para recibir recomendaciones. | Más información

Nombre del grupo de Auto Scaling

-

Administradas

falso

[Detalles](#)
[Estado y alarmas](#)
[Monitoreo](#)
[Seguridad](#)
[Redes](#)
[Almacenamiento](#)
[Etiquetas](#)

- ```
ssh -i maquina-david.pem ec2-user@18.205.23.164
```

```
ec2-user@ip-172-31-94-3 ~
```

```
+ v
```

```
Microsoft Windows [Versión 10.0.26100.3775]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\david>cd Downloads

C:\Users\david\Downloads>ssh -i maquina-david.pem ec2-user@18.205.23.164
The authenticity of host '18.205.23.164 (18.205.23.164)' can't be established.
ED25519 key fingerprint is SHA256:A2YBtftOqQAHgroPayAmII4rxERCofacEN6WhL4AtPQ.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '18.205.23.164' (ED25519) to the list of known hosts.
```

```

_##### Amazon Linux 2023
#_#####
#_####|
#_### \#/ https://aws.amazon.com/linux/amazon-linux-2023
 #/
 /_
 /_/_
 /_/_/_
 /_/_/_/_
 /_/_/_/_/_
/_/_/_/_/_/_
/_/_/_/_/_/_/_
/_/_/_/_/_/_/_/_
```

```
[ec2-user@ip-172-31-94-3 ~]$ |
```

[Aprende a instalar Docker en una instancia EC2 con Amazon Linux 2023 y Ubuntu](#)  
- DEV Community

### 1. Actualiza tu sistema

Es importante tener el sistema actualizado antes de instalar Docker:

```
sudo dnf update -y
```

### 2. Instala Docker

Amazon Linux 2023 utiliza el administrador de paquetes dnf. Instala Docker con:

```
sudo dnf install docker -y
```

```
[ec2-user@ip-172-31-94-3 ~]$ sudo dnf install docker -y
Last metadata expiration check: 0:01:48 ago on Mon May 5 10:11:09 2025.
Dependencies resolved.
=====
Package Architecture Version Repository Size
=====
Installing:
docker x86_64 25.0.8-1.amzn2023.0.3 amazonlinux 45 M
Installing dependencies:
container-selinux noarch 3:2.233.0-1.amzn2023 amazonlinux 55 k
containerd x86_64 1.7.27-1.amzn2023.0.2 amazonlinux 37 M
iptables-libs x86_64 1.8.8-3.amzn2023.0.2 amazonlinux 401 k
iptables-nft x86_64 1.8.8-3.amzn2023.0.2 amazonlinux 183 k
libcgroup x86_64 3.0-1.amzn2023.0.1 amazonlinux 75 k
libnetfilter_conntrack x86_64 1.0.8-2.amzn2023.0.2 amazonlinux 58 k
libnftnl x86_64 1.0.1-19.amzn2023.0.2 amazonlinux 30 k
libnftnl x86_64 1.2.2-2.amzn2023.0.2 amazonlinux 84 k
pigz x86_64 2.5-1.amzn2023.0.3 amazonlinux 83 k
=====
```

```
Installing : docker-25.0.8-1.amzn2023.0.3.x86_64 11/11
Running scriptlet: docker-25.0.8-1.amzn2023.0.3.x86_64 11/11
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket → /usr/lib/systemd/system/docker.socket.
Running scriptlet: container-selinux-3:2.233.0-1.amzn2023.noarch 11/11
Running scriptlet: docker-25.0.8-1.amzn2023.0.3.x86_64 11/11
Verifying : container-selinux-3:2.233.0-1.amzn2023.noarch 1/11
Verifying : containerd-1.7.27-1.amzn2023.0.2.x86_64 2/11
Verifying : docker-25.0.8-1.amzn2023.0.3.x86_64 3/11
Verifying : iptables-libs-1.8.8-3.amzn2023.0.2.x86_64 4/11
Verifying : iptables-nft-1.8.8-3.amzn2023.0.2.x86_64 5/11
Verifying : libcgroup-3.0-1.amzn2023.0.1.x86_64 6/11
Verifying : libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64 7/11
Verifying : libnftnl-1.0.1-19.amzn2023.0.2.x86_64 8/11
Verifying : libnftnl-1.2.2-2.amzn2023.0.2.x86_64 9/11
Verifying : pigz-2.5-1.amzn2023.0.3.x86_64 10/11
Verifying : runc-1.2.4-1.amzn2023.0.1.x86_64 11/11

Installed:
container-selinux-3:2.233.0-1.amzn2023.noarch
docker-25.0.8-1.amzn2023.0.3.x86_64
iptables-nft-1.8.8-3.amzn2023.0.2.x86_64
libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64
libnftnl-1.2.2-2.amzn2023.0.2.x86_64
runc-1.2.4-1.amzn2023.0.1.x86_64
containerd-1.7.27-1.amzn2023.0.2.x86_64
iptables-libs-1.8.8-3.amzn2023.0.2.x86_64
libcgroup-3.0-1.amzn2023.0.1.x86_64
libnftnl-1.0.1-19.amzn2023.0.2.x86_64
pigz-2.5-1.amzn2023.0.3.x86_64

Complete!
[ec2-user@ip-172-31-94-3 ~]$
```

### 3. Inicia y habilita Docker

Después de instalar Docker, debes iniciar el servicio y habilitarlo para que se ejecute automáticamente al reiniciar:

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

```
Complete!
[ec2-user@ip-172-31-94-3 ~]$ sudo systemctl start docker
[ec2-user@ip-172-31-94-3 ~]$ sudo systemctl enable docker
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
[ec2-user@ip-172-31-94-3 ~]$
```

### 4. Verifica la instalación

Asegúrate de que Docker esté instalado y en ejecución:

```
systemctl status docker
```

```
[ec2-user@ip-172-31-94-3 ~]$ docker --version
sudo systemctl status docker
Docker version 25.0.8, build 0bab007
● docker.service - Docker Application Container Engine
 Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
 Active: active (running) since Mon 2025-05-05 10:14:28 UTC; 1min 56s ago
 TriggeredBy: ● docker.socket
 Docs: https://docs.docker.com
 Main PID: 27921 (dockerd)
 Tasks: 7
 Memory: 38.5M
 CPU: 276ms
 CGroup: /system.slice/docker.service
 └─27921 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock --default-ulimit nofile=32768:65536

May 05 10:14:27 ip-172-31-94-3.ec2.internal systemd[1]: Starting docker.service - Docker Application Container Engine...
May 05 10:14:27 ip-172-31-94-3.ec2.internal dockerd[27921]: time="2025-05-05T10:14:27.635190966Z" level=info msg="Starting up"
```

## 5. Añade tu usuario al grupo Docker

Esto evita que tengas que usar sudo cada vez que ejecutas un comando de Docker:

```
sudo usermod -aG docker $USER
```

# Cierra sesión y vuelve a iniciar para aplicar los cambios.

```
[ec2-user@ip-172-31-94-3 ~]$ sudo usermod -aG docker $USER
[ec2-user@ip-172-31-94-3 ~]$
```

## 1. imagen mediante la sentencia `docker pull apache/kafka`

[illegible]

"Hacer pull" en Docker se refiere a la acción de **descargar una imagen de Docker desde un registro (registry)**. El registro más común y público es Docker Hub, pero también existen registros privados.

## Los puntos clave sobre docker pull:

- **Propósito:** El objetivo principal de docker pull es obtener una imagen que no está presente localmente en tu máquina. Necesitas la imagen para poder crear y ejecutar contenedores basados en ella.
- **Funcionamiento:** Cuando ejecutas el comando docker pull <nombre\_de\_la\_imagen>, el Docker daemon se conecta al registro configurado (por defecto, Docker Hub) y descarga la imagen especificada, junto con todas las capas y metadatos necesarios.

- **Sintaxis básica:** [docker image pull | Docker Docs](#)

```
docker pull [OPTIONS] NOMBRE_DE_LA_IMAGEN[:TAG|@DIGEST]
```

- **NOMBRE\_DE\_LA\_IMAGEN:** Es el nombre de la imagen que quieres descargar (ej: ubuntu, nginx, mysql).
- **TAG:** Es una etiqueta que especifica una versión particular de la imagen (ej: 16.04, latest, stable). Si no se especifica un tag, Docker asume :latest.
- **@DIGEST:** Es un identificador único y específico de una versión de la imagen, basado en su contenido. Usar el digest asegura que siempre obtendrás la misma versión, independientemente de los cambios que puedan ocurrir en las etiquetas.
- **OPTIONS:** Permiten modificar el comportamiento del comando pull (ej: autenticación, plataforma).

```
C:\Users\david>docker pull apache/kafka
Using default tag: latest
latest: Pulling from apache/kafka
e9e1a86a14df: Download complete
634d60101085: Download complete
c23d170f77ed: Download complete
a42500819bdc: Download complete
c6a83fedfae6: Download complete
1673c1a2cfa7: Download complete
0b502c76566e: Download complete
a39822f482a5: Download complete
e2d8f56b95eb: Download complete
c4c98316ca78: Download complete
Digest: sha256:c89f315cff967322c5d2021434b32271393cb193aa7ec1d43e97341924e57069
Status: Downloaded newer image for apache/kafka:latest
docker.io/apache/kafka:latest

What's next:
View a summary of image vulnerabilities and recommendations → docker scout quickview apache/kafka
```

## 2. Ejecutar docker(Run docker) Mediante la sentencia **docker run -d --name broker apache/kafka:latest**

```
[ec2-user@ip-172-31-46-244 ~]$ docker run -d --name broker apache/kafka:latest
1eb2d4d42c7f8b9b058702b1496cf9639784c5a1ad21708203b34e296d6a1b21
[ec2-user@ip-172-31-46-244 ~]$ |
```

Link a documentación de la instrucción [docker container run | Docker Docs](#)

Docker run es un comando fundamental en Docker que se utiliza para **crear y ejecutar contenedores a partir de una imagen**. En esencia, le dice a Docker que tome una imagen (que puede estar localmente o que Docker intentará descargar de un registro como Docker Hub si no la encuentra) y la ejecute como un proceso aislado llamado contenedor.

los aspectos clave de docker run son:

- **Propósito principal:** Ejecutar una instancia de una imagen, creando así un contenedor en funcionamiento.
- **Qué sucede al ejecutar docker run:**
  1. **Verifica la imagen localmente:** Docker busca la imagen especificada en tu sistema local.
  2. **Descarga la imagen (si es necesario):** Si la imagen no se encuentra localmente, Docker intenta descargarla del registro configurado (por defecto, Docker Hub). Esto es similar a un docker pull automático.
  3. **Crea un nuevo contenedor:** Se crea una instancia ejecutable de la imagen. Esto incluye una capa de escritura sobre la imagen de solo lectura, donde se pueden realizar cambios durante la vida útil del contenedor.
  4. **Configura el contenedor:** Se aplican las opciones que hayas especificado en el comando docker run (por ejemplo, nombre, puertos, volúmenes, variables de entorno).



5. **Inicia el contenedor:** Se ejecuta el proceso principal definido en la imagen (a través de CMD o ENTRYPOINT en el Dockerfile).

- **Sintaxis básica:**

**`docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`**

- **OPTIONS:** Permiten configurar diversos aspectos del contenedor, como el nombre (`--name`), mapeo de puertos (`-p`), montaje de volúmenes (`-v`), variables de entorno (`-e`), modo de ejecución (`detached -d`), etc.
- **IMAGE:** El nombre de la imagen que se utilizará para crear el contenedor (ej: `ubuntu`, `nginx:latest`, `mi-imagen:v1.0`).
- **COMMAND y ARG...:** Opcionalmente, puedes especificar un comando diferente al definido en la imagen para que se ejecute dentro del contenedor, junto con sus argumentos. Esto sobrescribe el CMD definido en el Dockerfile.

3. Abrir la shell in the broker container:

**`docker exec --workdir /opt/kafka/bin/ -it broker sh`**

```
[ec2-user@ip-172-31-46-244 ~]$ docker exec --workdir /opt/kafka/bin/ -it broker sh
/opt/kafka/bin $ |
```

`docker exec` es un comando de Docker que te permite **ejecutar comandos dentro de un contenedor que ya está en funcionamiento**. Es una herramienta muy útil para interactuar con procesos que se están ejecutando dentro de un contenedor, para depurar problemas, inspeccionar archivos o ejecutar tareas administrativas sin necesidad de detener o reiniciar el contenedor.

Aquí te explico los puntos clave sobre `docker exec`:

- **Propósito principal:** Ejecutar comandos adicionales dentro de un contenedor en ejecución.
- **Cómo funciona:** `docker exec` se conecta al espacio de nombres del contenedor especificado y ejecuta el comando que le indiques dentro de ese entorno aislado. Esto significa que el comando se ejecutará con el contexto del sistema de archivos, la red y otros recursos del contenedor.

- **Sintaxis básica:**

**`docker exec [OPTIONS] CONTAINER COMMAND [ARG...]`**

- **OPTIONS:** Permiten configurar cómo se ejecuta el comando dentro del contenedor (ej: modo interactivo, usuario).
- **CONTAINER:** El nombre o el ID del contenedor en el que quieres ejecutar el comando.
- **COMMAND:** El comando que quieres ejecutar dentro del contenedor (ej: `ls`, `bash`, `sh`, `cat`, `ps`).
- **ARG...:** Cualquier argumento adicional que necesite el comando.

- **Opciones comunes:**

- `-i` o `--interactive`: Mantiene la entrada estándar (`stdin`) abierta, lo que te permite interactuar con el comando (por ejemplo, si ejecutas una shell).
- `-t` o `--tty`: Asigna una pseudo-terminal (TTY), lo que es útil para comandos interactivos como shells, ya que proporciona funcionalidades como el historial de comandos y la finalización con tabulador.
- `-it`: La combinación de `-i` y `-t` es muy común para obtener una shell interactiva dentro del contenedor.

- `-u` o `--user`: Especifica el usuario con el que se ejecutará el comando dentro del contenedor (por defecto, se ejecuta como el usuario definido en la imagen o como root).
- `-w` o `--workdir`: Especifica el directorio de trabajo dentro del contenedor donde se ejecutará el comando.
- `--privileged`: Otorga privilegios extendidos al comando dentro del contenedor (generalmente no se recomienda por razones de seguridad).

#### 4. Creación de *topics*:

Crear un nuevo *topic* denominado 'pec-topic1-<NombreAlumno>' con dos particiones y con factor de replicación 1.

Link a documentación explicativa [Apache Kafka](#)

Se crea mediante la sentencia `./kafka-topics.sh --bootstrap-server localhost:9092 --create --topic david-carvajal`

```
/opt/kafka/bin $./kafka-topics.sh --bootstrap-server localhost:9092 --create --topic david-carvajal
Created topic david-carvajal.
/opt/kafka/bin $ |
```

Si quisiéramos comprobar los *topics* que hemos creado, podemos obtener un listado mediante el parámetro `--list`:

`./kafka-topics.sh --bootstrap-server localhost:9092 --list`

```
/opt/kafka/bin $./kafka-topics.sh --bootstrap-server localhost:9092 --list
david-carvajal
/opt/kafka/bin $ |
```

¿Sería posible crearlo en el entorno que tienes con un factor de replicación de 2?

¿Por qué?

[Introducción a Apache Kafka — AprenderBigData.com | by Oscar Fmdc | Medium](#)

**Comando para crear un topic con particiones y factor de replicación:**

`./kafka-topics.sh --bootstrap-server localhost:9092 --topic david-carvajal-replications --create --partitions 3 --replication-factor 2`

```
/opt/kafka/bin $./kafka-topics.sh --bootstrap-server localhost:9092 --topic david-carvajal-replications --create --partitions 3 --replication-factor 2
Error while executing topic command : Unable to replicate the partition 2 time(s): The target replication factor of 2 cannot be reached because only 1 broker(s) are registered.
[2025-05-05 20:56:13.864] ERROR org.apache.kafka.common.errors.InvalidReplicationFactorException: Unable to replicate the partition 2 time(s): The target replication factor of 2 cannot be reached because only 1 broker(s) are registered.
(org.apache.kafka.tools.TopicCommand)
/opt/kafka/bin $ |
```

Nos devuelve error porque solo tenemos un broker. Por tanto la respuesta es NO PORQUE solo tenemos un BROKER.

- Crea un productor que empiece a pasar información desde la línea de comandos al *topic* creado anteriormente y simular el envío de información.

### Creación de un productor

`./kafka-console-producer.sh`

Este comando lleva a documentación, donde se pueden ver los parámetros obligatorios:

```
/opt/kafka/bin $./kafka-console-producer.sh
This tool helps to read data from standard input and publish it to Kafka.
Option Description
----- -
--batch-size <Integer: size> Number of messages to send in a single
 batch if they are not being sent
 synchronously. please note that this
 option will be replaced if max-
 partition-memory-bytes is also set
 (default: 16384)
--bootstrap-server <String: server to REQUIRED: The server(s) to connect to.
connect to> The broker list string in the form
 HOST1:PORT1,HOST2:PORT2.
--compression-codec [String: The compression codec: either 'none',
```

Para crear el productor ejecutar lo siguiente:

`./kafka-console-producer.sh --bootstrap-server localhost:9092 --topic david-carvajal`

>escribir

>escribir

>escribir

CTRL-C para salir

```
/opt/kafka/bin $./kafka-console-producer.sh --bootstrap-server localhost:9092 --topic david-carvajal
>hola
>clase
>estoy
>en aws
>
```

- Creación de consumidores:
  - Crear un consumidor que sea capaz de leer toda la información que contenga el *topic* creado.

### Creación de un consumidor

En este punto, vamos a ver cómo consumir la información que se ha dejado en el *topic* de Kafka.

Como siempre para ver la documentación:

`./kafka-console-consumer.sh`

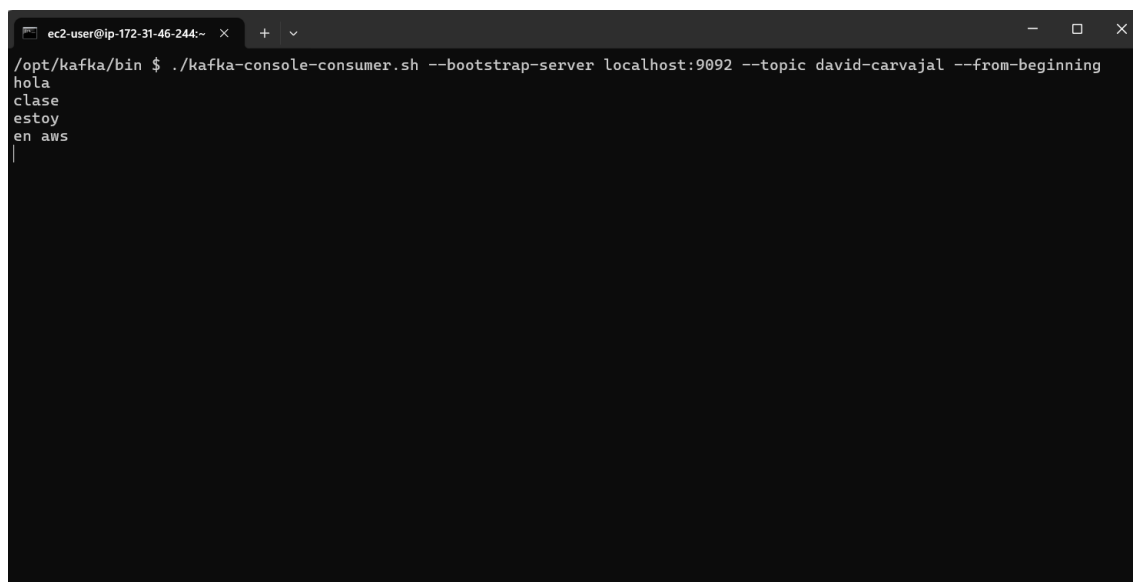
```
>^C/opt/kafka/bin $./kafka-console-consumer.sh
This tool helps to read data from Kafka topics and outputs it to standard output.
Option Description

--bootstrap-server <String: server to connect to> REQUIRED: The server(s) to connect to.
--consumer-property <String: consumer_prop> A mechanism to pass user-defined properties in the form key=value to the consumer.
--consumer.config <String: config file> Consumer config properties file. Note that [consumer-property] takes precedence over this config.
--enable-systest-events Log lifecycle events of the consumer in addition to logging consumed messages. (This is specific for system tests.)
--formatter <String: class> The name of a class to use for formatting kafka messages for display. (default: org.apache.kafka.tools.consumer.DefaultMessageFormatter)
--formatter-config <String: config> Config properties file to initialize
```

Ejecutamos un consumidor:

Para leer todos los msgs, y no solo los que se manden una vez se ha arrancado, se aplica el parámetro ‘--from-beginning’.

```
./kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic david-carvajal --from-beginning
```



```
ec2-user@ip-172-31-46-244:~ $./kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic david-carvajal --from-beginning
hola
clase
estoy
en aws
```

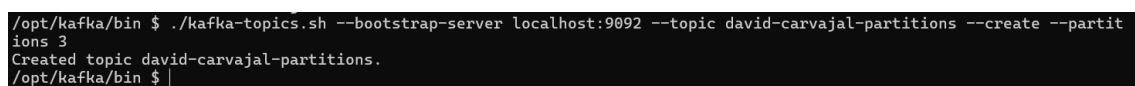
Crear un grupo de consumidores con el suficiente número de ellos para que cada uno lea de una de las particiones del *topic*.

[Apache Kafka. Elementos y ejemplos con Python. - IABD](#)

[Kafka consumer group](#)

Primero crearemos un *topic* que contenga tres particiones:

```
./kafka-topics.sh --bootstrap-server localhost:9092 --topic david-carvajal-partitions --create --partitions 3
```



```
/opt/kafka/bin $./kafka-topics.sh --bootstrap-server localhost:9092 --topic david-carvajal-partitions --create --partitions 3
Created topic david-carvajal-partitions.
/opt/kafka/bin $
```

Si comprobamos el estado del *topic* mediante:

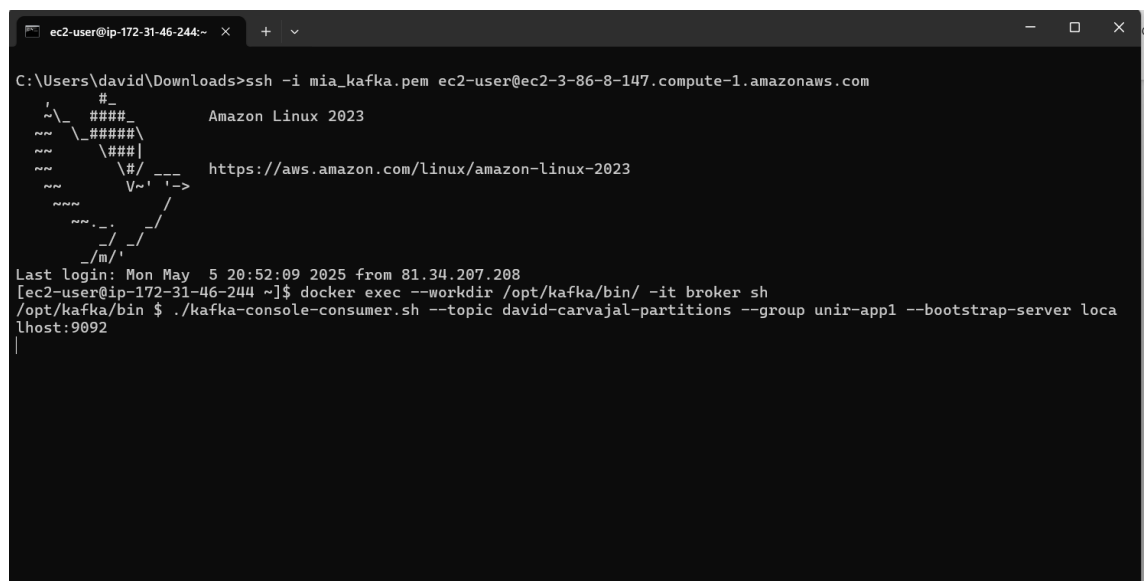
```
./kafka-topics.sh --describe --topic david-carvajal-partitions --bootstrap-server localhost:9092
```

Obtendremos la siguiente información:

```
/opt/kafka/bin $./kafka-topics.sh --describe --topic david-carvajal-partitions --bootstrap-server localhost:9092
Topic: david-carvajal-partitions TopicId: yKOUFQjxSQKKeWofgH4DsQ PartitionCount: 3 ReplicationFactor: 1 C
onfigs: segment.bytes=1073741824
 Topic: david-carvajal-partitions Partition: 0 Leader: 1 Replicas: 1 Isr: 1 Elr: LastKnow
nElr:
 Topic: david-carvajal-partitions Partition: 1 Leader: 1 Replicas: 1 Isr: 1 Elr: LastKnow
nElr:
 Topic: david-carvajal-partitions Partition: 2 Leader: 1 Replicas: 1 Isr: 1 Elr: LastKnow
```

A continuación, en dos pestañas diferentes, vamos a crear dos consumidores que pertenezcan al mismo grupo de consumidores (para ello debemos entrar en esas dos nuevas pestañas en EC2 de AWS y en la shell de Apache Kafka con docker exec)

```
./kafka-console-consumer.sh --topic david-carvajal-partitions --group unir-app1 --bootstrap-server localhost:9092
```



```
ec2-user@ip-172-31-46-244:~$ ssh -i mia_kafka.pem ec2-user@ec2-3-86-8-147.compute-1.amazonaws.com
C:\Users\david\Downloads>ssh -i mia_kafka.pem ec2-user@ec2-3-86-8-147.compute-1.amazonaws.com
#####
 _ _ _ _ _
 / _ _ _ _ \
 / _ _ _ _ \
 / _ _ _ _ \
 / _ _ _ _ \
 / _ _ _ _ \
/_ _ _ _ _ \
/m/

Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Mon May 5 20:52:09 2025 from 81.34.207.208
[ec2-user@ip-172-31-46-244 ~]$ docker exec --workdir /opt/kafka/bin/ -it broker sh
/opt/kafka/bin $./kafka-console-consumer.sh --topic david-carvajal-partitions --group unir-app1 --bootstrap-server localhost:9092
```

Ventana2 (DONDE DEBEMOS EJECUTAR EL EXEC para entrar en shell Kafka

```
./kafka-console-consumer.sh --topic david-carvajal-partitions --group unir-app1 --bootstrap-server localhost:9092
```



Y vemos que se leen en el grupo de consumidores , en cada consumidor

```
ec2-user@ip-172-31-46-244:~
/opt/kafka/bin $./kafka-console-consumer.sh --topic david-carvajal-partitions --group unir-app1 --bootstrap-server localhost:9092
Este es un mensaje para el usuario 1
Este es un mensaje sobre el producto A
Un evento más para el usuario 1
|
```

```
ec2-user@ip-172-31-46-244:~
/opt/kafka/bin $./kafka-console-consumer.sh --topic david-carvajal-partitions --group unir-app1 --bootstrap-server localhost:9092
Otro mensaje para el usuario 2
Información detallada del producto B
|
```

Explicación de esta sentencia

- <host\_kafka>:<puerto\_kafka>: Reemplaza esto con la dirección y el puerto de tu broker de Kafka (por ejemplo, localhost:9092 o el nombre del servicio Kafka dentro de tu red Docker si estás usando Docker Compose, como kafka:9092).
- <nombre\_del\_topic>: Reemplaza esto con el nombre del topic al que quieres enviar mensajes.

- `--property parse.key=true`: Esta propiedad le dice al productor que espere un formato de entrada que incluya una clave.
- `--property key.separator=,`: Esta propiedad define el carácter que separará la clave del valor en tus mensajes. En este ejemplo, estamos usando una coma (,). Puedes usar cualquier otro carácter que desees (por ejemplo, :, -, etc.).

¿Cómo se escriben mensajes con clave y valor?

Después de ejecutar el comando, la consola esperará a que escribas los mensajes.

Debes escribir cada mensaje en la siguiente forma:

`<clave>,<valor>`

Y si creamos varios mensajes en el productor, veremos cómo van llegando de manera alterna a los diferentes consumidores:

Hay que tener en cuenta las siguientes consideraciones:

- **No especificar clave:** Si no especificas una clave de partición al producir mensajes, Kafka utilizará una estrategia de round-robin para distribuir los mensajes entre las particiones (y por lo tanto, entre los consumidores del grupo).
- **Usar claves diferentes:** Si necesitas mantener el orden de los mensajes relacionados, asegúrate de usar diferentes claves de partición para diferentes conjuntos de mensajes que no necesiten estar en el mismo orden. Una estrategia común es usar un ID de usuario o un ID de sesión como clave.